Calcio Champagne

Project Report

A reproducible football-data system: acquisition, analytical modelling, infrastructure measurements and a browser application.

Author: Atai Keneshbekov · individual portfolio project
Evidence snapshot: 6 November 2026
Repository: Calcio-Champagne-prediction-automated-system
Submission status: technical evidence documented; source-licence and independent-validation items remain open

REPORT STRUCTURE

Contents

1. Abstract · 3
2. Project question, motivation and objectives · 4
3. Requirements and scope traceability · 5
4. Data sources and provenance · 6
5. Acquisition, validation and ETL · 7
6. Relational model and derived measures · 8
7. System architecture and runtime · 9
8. Architecture in UML: components and deployment · 10
9. Architecture in UML: the data model · 11
10. Architecture in UML: the startup sequence · 12
11. Attacking index and descriptive analysis · 13
12. Prediction target and feature construction · 14
13. Temporal evaluation and model diagnostics · 15
14. Defensive metrics, match context and xG boundary · 16
15. Application and interface · 17
16. Verification strategy and results · 18
17. Infrastructure measurement protocol · 19
18. Growth, memory and storage results · 20
19. Results and interpretation · 21
20. Risks, limitations and responsible claims · 22
21. Project evolution and contribution evidence · 23
22. Conclusion · 24
23. References · 25
24. Reproduction and appendix: command route · 26

> **Reading note.** This report follows the formal structure of a course project report. It describes observed repository behavior and dated measurements; planned, descriptive and unresolved items are labeled as such.

Primary technical records are maintained in README.md and the linked docs/ directory. The report is a concise submission narrative, not a replacement for the code, raw experiment records or source manifest.

Abstract

Calcio Champagne is a reproducible football-data system built around a practical question: does recorded transfer spending accompany an improvement in a club's next-season attacking output? A second question asks whether that output tracks the same season's league finish. The project combines pinned CSV acquisition, PostgreSQL processing, an evaluated regression model, an HTTP API and an editorial browser interface.

The central measure, the Champagne Index, is (recorded goals + assists) divided by the source's team-season match-count proxy. It is intentionally treated as an attacking-output index, not as expected goals, a tactical rating or a causal measure of transfer value. Optional completed-match data supplies actual league goals, goals conceded, goal difference and direct meetings for five leagues.

In 38,531 eligible consecutive team-season pairs, incoming spend has essentially zero linear association with the next-season change in the index ($r = -0.0036$). In 388 adequately covered top-five club-seasons, the index is more strongly associated with same-season final position ($r = -0.7933$), an explicitly retrospective relationship. A chronological RandomForest reaches $R^2 = 0.1271$ and MAE = 0.4856 on 5,585 later observations; predicting no change yields MAE = 0.5114 on those same observations.

> **Project conclusion.** The result supports a modest predictive signal over a simple baseline, not a strong forecasting product or a claim that spending causes improvement. Data boundaries, sample coverage and source licensing materially limit the conclusions.

The repository also records repeated system measurements, API/browser checks, source provenance and an examiner-preparation package. The primary Transfermarkt-derived source has no confirmed reuse licence, so that course requirement remains open.

Project	Calcio Champagne prediction automated system	**Evidence snapshot**	6 November 2026
Course context	LUISS · Computational Infrastructures for Data Science		
Report scope	Data engineering, analytics, software system, verification and limitations		

Project question, motivation and objectives

Motivation

Football discussions often treat transfer budgets, attacking output and league success as if one automatically explains another. This project converts that discussion into a reproducible data workflow. It separates three units that are easy to conflate: player-season performance records, club-season attacking output and completed match results. Each answers a different question and has a different denominator.

Research questions

1. Does incoming recorded transfer spending predict the change in a club’s attacking index in its next consecutive season?
2. How closely does the attacking index accompany the club’s final position in the same completed season?
3. Can a small set of season-level features improve on predicting zero change under a chronological holdout?
4. Can the source and pipeline be run, checked and explained as a complete data system?

Objectives and acceptance evidence

Objective	Evidence
Acquire inputs reproducibly	Pinned manifest, ordered-header/hash checks, atomic download, offline reuse
Preserve valid analytical units	Canonical club/season/competition keys; consecutive targets; coverage gates
Evaluate prediction fairly	Time-ordered holdout and zero-change baseline on identical rows
Expose results responsibly	Documented API contracts, source coverage, visible caveats and export
Measure infrastructure	Three-run startup/query/resource protocol and partial-data growth tests

The system is an educational and retrospective analysis. It does not recommend transfers, predict match outcomes, estimate player quality or claim causal impact. These boundaries shape both the model design and the wording shown in the interface.

Requirements and scope traceability

The course brief emphasizes a runnable data system, database rationale, clear source acquisition, a stranger-readable README and measured performance. Machine-learning and web-interface additions are optional enhancements; the relational load/process/measure workflow remains the course core.

Requirement	Implementation	Evidence / status
Reproducible source acquisition	Python loader and pinned source manifest	Header/hash checks and temporary-directory acquisition exercise
Database with justification	PostgreSQL 16, fixed-schema CSV imports, relational joins	Schema, COPY ingestion, indices and named persistent volume
One-command startup	Docker Compose health and completion dependencies	Cached-source start averaged 31.10 s in the measured snapshot
Performance numbers	Query, growth and full-system measurement scripts	Three repetitions, hardware and limitations documented
Four real contributors	Not satisfied by this solo portfolio exercise	README explicitly states account/person distinction
Clear primary-data licence	Not established	Source revision has no explicit licence file; permission or replacement remains necessary

Scope decisions

The core source has broad historical player, transfer and club records. Optional match results cover five leagues and are used only when present. The application distinguishes missing enrichment from a valid zero value. No large distributed stack was introduced: the observed working set and dataset scale did not justify Spark, Kubernetes or a cloud dependency.

> **Submission boundary.** A polished interface, accounts, role cards or commit counts do not establish a real group contribution. The repository identifies this as one author’s portfolio exercise and leaves course actions requiring other people clearly open.

Requirement mapping, page references and remaining human actions are maintained in `docs/COURSE_REQUIREMENTS.md` and `docs/WORK_PACKAGES.md`.

Data sources and provenance

Primary source

The main snapshot contains eleven CSV tables from `salimt/football-datasets`, pinned to revision `da9227e1f42baf74561324290dbf1ad2e3c00edd`. The audited local copy contains 5,743,186 rows. Ten local files match the pinned upstream objects exactly. One legacy team-competition metadata table has a different schema and row count; the loader documents and normalizes the pinned upstream version for a fresh checkout. That table does not feed the central index or prediction model.

Relevant tables include player profiles, player performances, transfers, market values, team details, team hierarchy and injuries. Performance rows contain goals, assists, cards, team-season matches proxy, goalkeeper conceded goals and clean sheets. Transfer fees are highly incomplete: approximately 95% of rows have a recorded value of zero, including loans and undisclosed fees. Therefore zero is not interpreted as a confirmed free transfer in all cases.

Optional match enrichment

Twenty-five manually downloaded football-data.co.uk files cover England, Spain, Italy, Germany and France from 2021/22 through 2025/26. The prepared snapshot has 8,908 completed fixtures. Shots, shots on target, corners, cards, half-time scores and odds are present for 8,907 fixtures. The files are excluded from project archives.

Source	Grain / coverage	Use	Known boundary
Transfermarkt-derived pinned CSVs	Player/transfer/team season; 11 tables	Index, spend, roster inputs	Licence unresolved; zero fees may mean missing/undisclosed
Football-Data.co.uk	Match-level, 5 leagues × 5 seasons	Standings, GF/GA, shots, meetings	Manual download; not redistributed; coverage ends at this scope

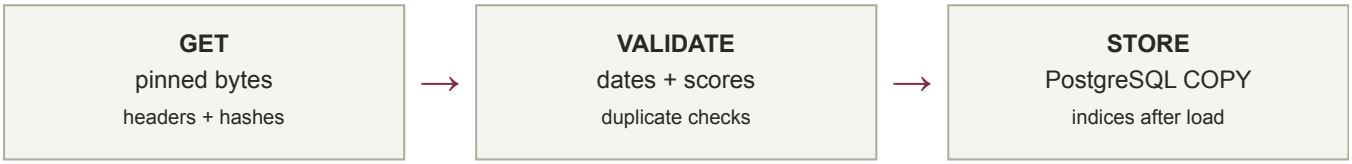
Football-Data’s published policy describes free CSV/Excel files for personal or academic research, with manual download or light automation. That policy does not resolve the separate primary source’s licence. Public availability is not treated as open-data permission.

Acquisition, validation and ETL

The loader is an explicit GET–STORE boundary. `loader/get_data.py` checks every expected file's ordered headers and pinned checksum. Missing inputs can be downloaded from the pinned revision without a login. A download is written to a temporary path and only atomically installed after validation; an interrupted or mismatched transfer cannot replace a known-good file. Existing verified inputs can be reused offline.

`loader/load_data.py` creates the relational schema, loads the eleven tables using PostgreSQL bulk `COPY`, then adds query indices. It validates match filenames, dates, nonnegative integer scores and whether the stated result agrees with the score. Unplayed fixtures are excluded. Both two-digit and four-digit date years are accepted. Duplicate fixtures load once; contradictory score duplicates abort before replacing existing rows.

Match statistics are optional nullable columns. Additive migrations preserve earlier match rows and schema updates avoid dropping a loaded table. Removing optional source files clears stale imported match rows on a subsequent load so the application cannot silently present obsolete standings.



Canonical identity

Source club aliases are mapped to canonical identifiers before match joins. The analytical key is club × season × competition; names are presentation fields. This prevents alternate spellings from multiplying rows. The pipeline constructs unique club-season inputs before attaching competition rows, which is necessary for safe market-value lags.

Acquisition checks use fake HTTP responses for error paths. A separate temporary-directory run downloaded one public file, verified its hash and header, and installed it without modifying the prepared project snapshot.

Relational model and derived measures

PostgreSQL fits this workload because the inputs have stable tabular schemas and recurring joins by player, club, competition and season. Primary tables retain source-level facts; SQL views and pipeline outputs express analytical facts at a declared grain. A named Docker volume preserves database state across container rebuilds.

Central analytical grain

$$\text{club_id} \times \text{season} \times \text{competition_id} \rightarrow \text{one output row}$$

The main view aggregates player performances to a club-season-competition record. The Champagne Index is the sum of recorded goals and assists divided by the source match-count proxy. Goals and assists are counted events, so an assisted goal contributes to each player statistic in the source convention; this is not a match-by-match team score reconstruction.

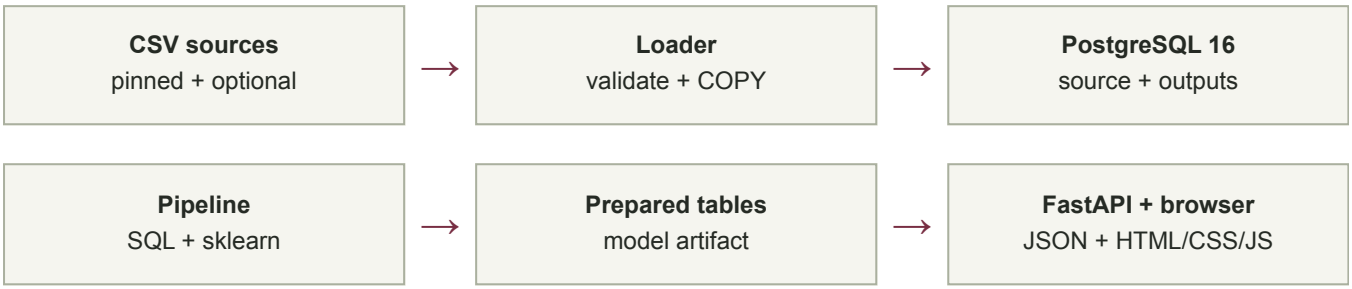
Incoming transfer spend is aggregated separately by receiving team and season. Market value is reduced to one value per player and season before comparing the club's current roster with the exact preceding season. Only then are competition rows joined. This avoids a common duplication bug where one club is repeated across competitions before calculating the lag.

Derived field	Definition	Interpretation
Champagne Index	(goals + assists) / source matches	Attacking output proxy per source match
Delta index	Next consecutive season index − current index	Prediction target; not level or rank
Discipline Index	(yellow + second yellow + direct red) / source matches	Coverage-sensitive card rate
Goal difference	Completed-match GF − GA	Actual scoreline aggregate, separate from index

Reserve and youth teams are excluded from model fitting through the team hierarchy table. Coverage thresholds differ by competition: top-five league pairs require at least 30 matches in both seasons; other competitions require five. A target is never formed by jumping across a missing year.

System architecture and runtime

The default deployment consists of PostgreSQL, a one-shot loader, a one-shot analytics pipeline and a FastAPI application. Docker Compose waits for database health, then successful loader completion, then successful pipeline completion before starting the application. An optional measurement service runs on demand. The original Streamlit interface remains available behind a separate profile but is not the primary product.



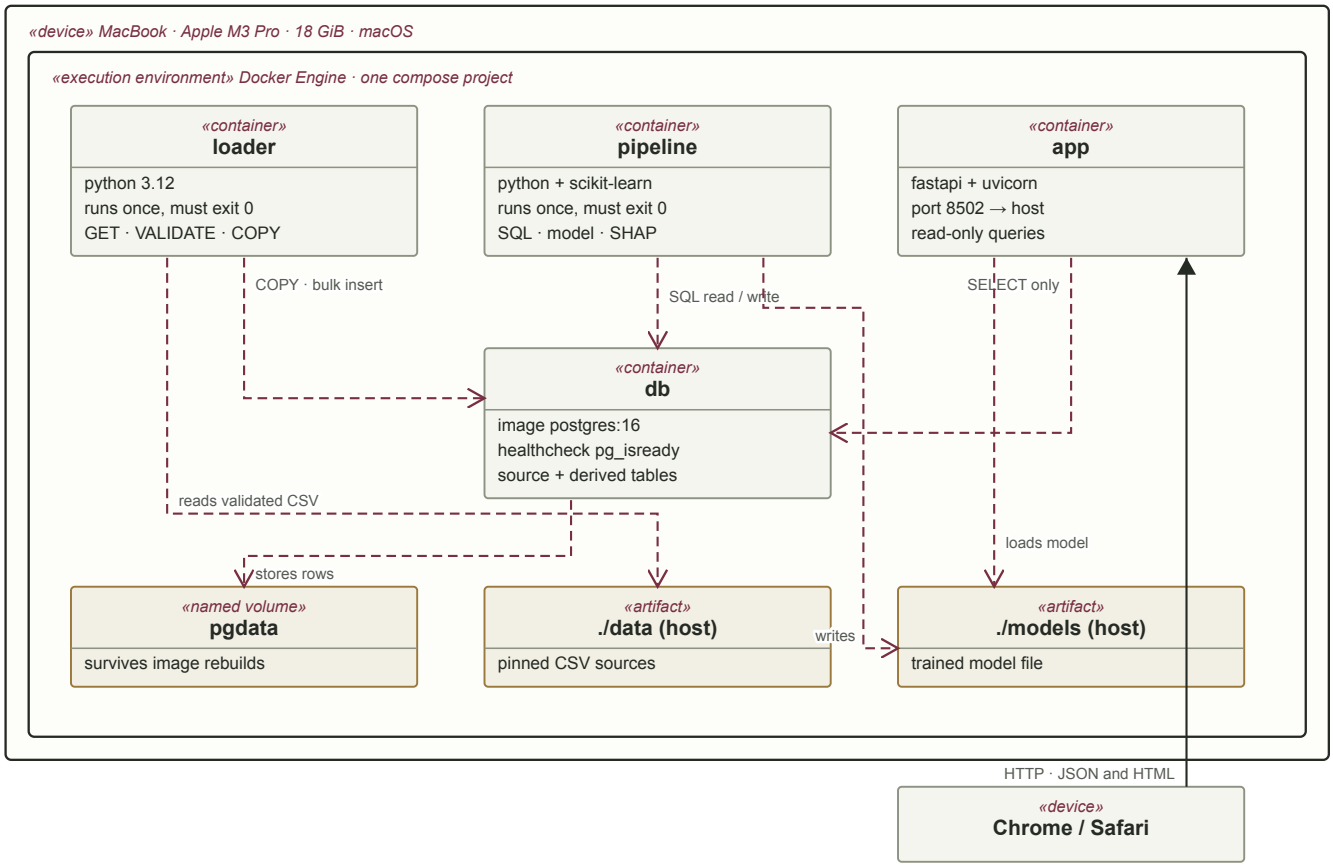
The app serves a buildless static frontend with native SVG charts. Runtime requires no JavaScript compilation, CDN assets, remote fonts or frontend framework. Read queries use bound parameters, timeouts and read-only transaction defaults; raw player tables are not exposed through public endpoints. Model inference loads a local trusted artifact and reloads it if the file timestamp changes.

Source files, model weights, credentials and measurement artifacts are excluded from the Docker build context. CSVs and models use explicit host mounts; the database uses a named volume. The app runs as a non-root user. These choices make the deployment simple to inspect, but the project remains a local single-machine deployment rather than a production high-availability service.

The main API includes health, metadata, season analysis, club profiles, comparisons, head-to-head, Lab configuration/prediction, CSV export and project evidence. `/api/health` reports missing tables and empty outputs; optional model availability is reported separately from data readiness.

Architecture in UML: components and deployment

This diagram is a UML component and deployment view: it shows what actually runs on the machine and what each part is allowed to depend on. Read the notation as follows. A rectangle is an element of the system. The italic label in guillemets, such as *«container»*, is a stereotype: it names the kind of element rather than its identity. A dashed arrow with an open head is a dependency, read as “uses”. A solid arrow with a filled head is a flow of control or data. The amber rectangles are artifacts: the three places where state is kept between runs.



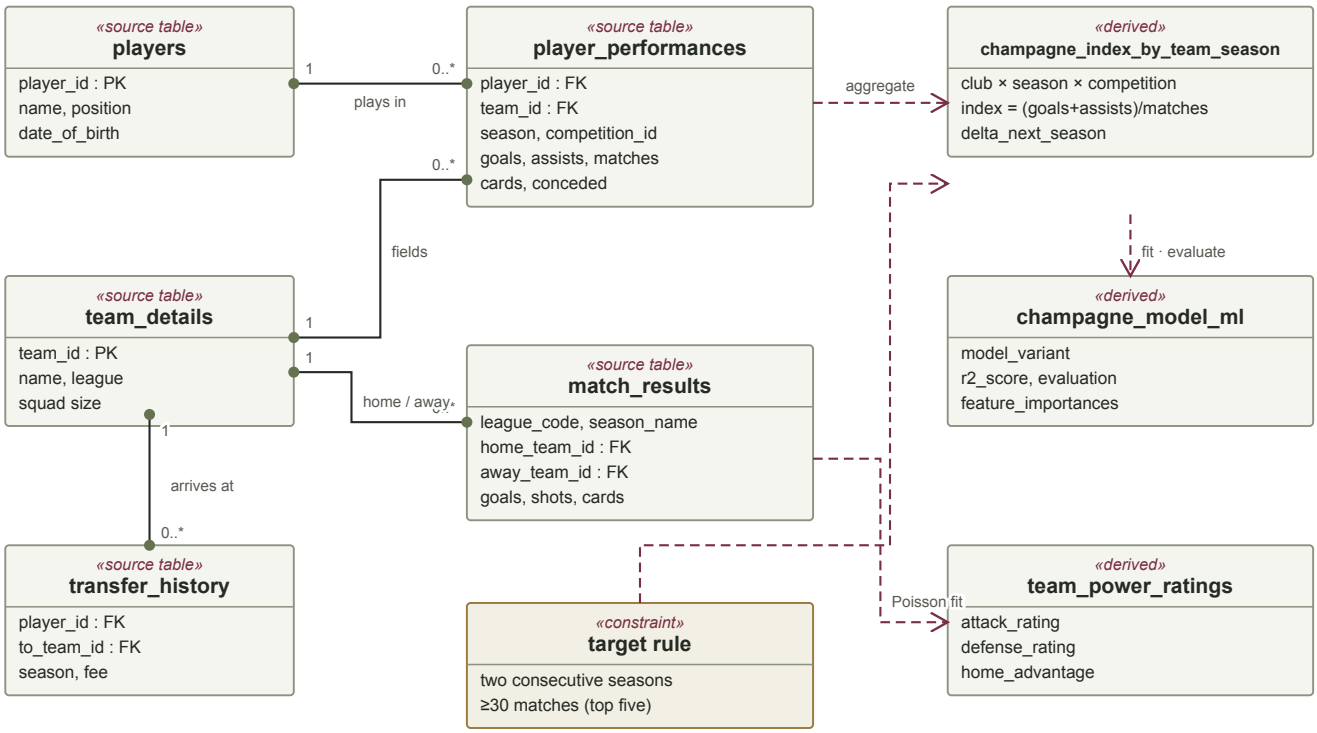
Three properties of the design are visible directly in the figure. First, every dependency points toward the database and never back: the loader and the pipeline write, the application only reads. Second, the application reads prepared tables rather than raw source rows, which is why the browser never receives millions of records. Third, state lives in exactly three places — a named volume for the database, a host mount for the source CSVs and a host mount for the model artifact — so rebuilding an image never destroys loaded data.

The figure also shows what is deliberately absent. There is no message broker, no distributed scheduler, no cloud service and no second node. At this data scale the observed working set did not justify them, and each one would have added a failure mode that a reader of this report could not inspect.

> **How to use this in the defense.** Point at one arrow and say what travels along it: a validated CSV file, a bulk `COPY` of rows, a SQL query, a JSON response. A diagram whose arrows you can name in concrete terms is evidence; a diagram of unexplained boxes is decoration.

Architecture in UML: the data model

This is a UML class diagram of the data the system stores. Each box is a table. The upper compartment holds the name, the lower compartment the attributes that matter for the analysis; PK marks a primary key, which identifies a row uniquely, and FK marks a foreign key, which points at a row in another table. A line ending in filled dots is an association, and the numbers at its ends are multiplicities: 1 on one side and `0..*` on the other reads as “one club has zero or more performance rows”. The olive boxes carry the stereotype *«derived»*: they are computed by the pipeline and rewritten completely on every run, so they are never edited by hand.

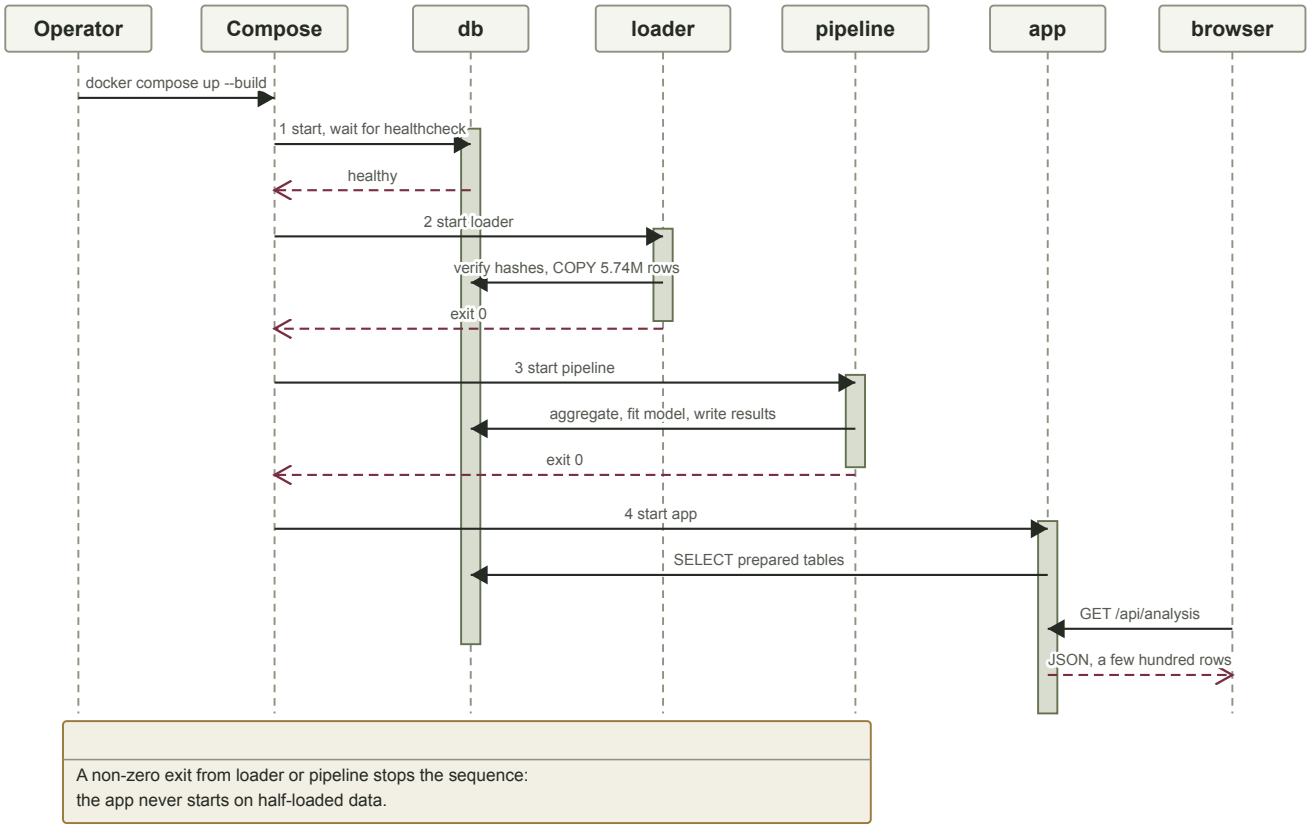


The important structural decision is the analytical grain, shown by the aggregation dependency: one output row is one club in one season in one competition. Everything upstream is finer than that (one row per player per season) and everything downstream is coarser or equal. Keeping that single grain is what prevents the most common bug in this kind of project, where a club that appears in three competitions is counted three times before a per-season value is computed.

The constraint box is part of the model, not a footnote: a prediction target is only formed from two genuinely consecutive seasons with adequate match coverage in both. Without that rule the next recorded row for a club can be several years later, and the resulting “next season” target would silently measure something else.

Architecture in UML: the startup sequence

This UML sequence diagram shows the order in which the system comes up after a single command. Each labelled box at the top is a participant, and the dashed vertical line below it is its lifeline, with time running downward. A solid arrow is a call or an action, a dashed arrow with an open head is a return, and a grey bar on a lifeline marks the period in which that participant is doing work. The numbers on the messages are the four gates Compose enforces.



The sequence encodes the project's main safety rule: each stage starts only after the previous one has finished successfully. The database must report healthy before the loader runs, the loader must exit with status zero before the pipeline runs, and the pipeline must exit with status zero before the application starts. A failure therefore surfaces as a stopped startup rather than as a web page showing half-loaded data.

The measured 31.10-second startup is this exact sequence, averaged over three runs with prepared sources and existing images: database health, source import, model training and the application becoming ready. The loader mean of 12.53 s and the pipeline mean of 9.70 s are the two grey bars in the middle of the figure, which is why those two numbers add up to most of the total.

> **Reading the figure as a timeline.** The browser interaction at the bottom is a separate, repeatable event: it happens after startup and takes milliseconds, not seconds. Do not quote the startup number when asked how fast a page loads — they measure different things.

Attacking index and descriptive analysis

The project separates transfer-spending association from same-season performance association. For the first question, it measures incoming recorded spend against next consecutive season change in Champagne Index. The Pearson correlation is r = −0.003613, p = 0.478242, n = 38,531 eligible pairs. This is close to zero and provides no evidence of a useful linear relationship in this snapshot.

For the second question, the analysis compares Champagne Index with final league position. Among 388 adequately covered top-five club-seasons, r = −0.793285. The negative sign reflects rank convention: a smaller numerical position means a better finish. The analysis excludes partial 2025/26 performance data paired with a full-season final table, requiring adequate match coverage and at least 30 played matches.

Association	Sample	Result	Meaning
Spend → next index change	38,531 eligible pairs	r = −0.0036; p = 0.478	No useful linear association detected
Index ↔ same-season position	388 covered club-seasons	r = −0.7933	Higher index accompanies lower (better) position number
Defender spend → next conceded-rate change	38,531 pairs	r = +0.0040; p = 0.427	No useful linear association detected

Correlation is not causation. Club strength, competition, player availability, retained squads, tactical choices, reporting practices and financial constraints are not randomized. The position relationship is also contemporaneous: it describes a completed season and is not a prospective forecast.

The interface labels sample size and view scope, and keeps system-level evaluation distinct from correlations calculated on a selected league/season. A small p-value would not by itself imply a meaningful effect; practical magnitude and denominator remain central.

Prediction target and feature construction

One observation is a club-season-competition row. The target is next season's Champagne Index minus the current season's index. A valid target requires a consecutive season in the same competition and sufficient source match coverage in both seasons. Reserve/youth squads are removed. This avoids leakage from future rows and prevents promotion/relegation or missing-season gaps from masquerading as the next season.

The baseline model uses five features: current Champagne Index, total incoming transfer spend, attacking-player spend, average squad age and change in average player market value. The first four represent current output, investment and squad composition. Market-value change is derived after deduplication to the player-season level and exact previous-season join. Missing Lab inputs are surfaced rather than filled with a hidden median.

The scikit-learn model is a RandomForestRegressor with 200 trees, maximum depth 6 and fixed seed 42. Preprocessing and feature order are persisted with the model artifact. Hyperparameters are fixed for this report; no nested cross-validation or large hyperparameter search is claimed.

> **Intended use.** The Lab is a retrospective educational scenario explorer. It compares a baseline input row with a modified row using the separately saved all-data model. It does not simulate a real transfer window, establish a causal spending effect or promise future league position.

A previous random row split mixed seasons and could place later observations in training while testing earlier ones. That score is retained only as historical context and is not compared directly with the corrected temporal evaluation. The prediction unit, target and time boundary now define the claim before the metric is interpreted.

Input	What it represents	Potential limitation
Current index	Recent recorded attacking output	Source match-count proxy
Total / attacking spend	Incoming fee totals	Undisclosed amounts recorded as zero
Average age	Distinct roster's season age	Roster coverage and source identity
Market-value change	Exact year-over-year roster value shift	Value estimate is not realized performance

Temporal evaluation and model diagnostics

The evaluation reserves the latest seasons representing roughly one fifth of eligible observations as a chronological test set. All competitions from a season remain on the same side of the boundary. Seasons starting in 2021 and later form the current held-out portion. The evaluation model fits only earlier seasons; the Lab model is a separate model trained on all eligible rows.

Protocol / model	Train rows	Test rows	R^2	MAE
5-input RandomForest	16,074	5,585	0.127115	0.485593
Zero-change baseline	—	Same 5,585	−0.000183	0.511364
Augmented feature group	Same time cutoff	Same 5,585	0.1502	0.4788

Adding goalkeeper conceded rate, discipline and net-spend features together improves R^2 from 0.1271 to 0.1502 and lowers MAE from 0.4856 to 0.4788. This is a modest improvement on the same holdout. Because the features were added as a group, the comparison does not isolate which single addition accounts for the change.

Held-out SHAP values summarize how the fitted tree ensemble allocates predictions across features; the leading global mean absolute value is Champagne Index at 0.2054 index points. SHAP is an explanation of this fitted model, not causal importance or a guarantee of stable feature effects across datasets. Model output remains much less certain than a rank or a bookmaker forecast might imply.

> **Interpretation.** MAE is measured in index points. The roughly 5% baseline reduction is not “95% accuracy”; $R^2 = 0.127$ means the model explains only a modest share of held-out variance under this particular split.

Evaluation evidence is stored in the `champagne_model_ml` output and exposed separately from the all-data inference artifact. Recompute the pipeline after source or feature changes and re-check the holdout rather than carrying old scores forward.

Defensive metrics, match context and xG boundary

The match enrichment supports actual club-season goals for (GF), goals against (GA) and goal difference (GD = GF − GA). Attack rank orders GF per completed match descending; defense rank orders GA per match ascending. Ranks are scoped to one selected league-season, and ties share a rank. These are transparent score-derived rankings, not proprietary ratings.

On 8,907 of 8,908 matches, the source provides shots, shots on target and corners. Comparisons show these observed statistics with their match coverage. Shot accuracy is shots on target divided by shots; goals-per-shot-on-target is a conversion rate, not chance quality. Direct head-to-head aggregates preserve the perspective of home and away fixtures and remain separate from whole-season records.

The source has no provider xG field. The SOT-based estimate multiplies a club's shots on target by the league-season-wide goals-per-shot-on-target rate. It contains no shot location, distance, body part, pressure, goalkeeper position or possession context. The UI labels it as an experimental volume proxy, never measured xG.

A two-parameter Poisson model is fit independently for each adequately covered league-season: home and away scores are Poisson rates based on team attack, opponent defense and a shared home-advantage term. A season requires at least 50 matches. This produced 484 team-season ratings over 25 league-seasons. The ratings are fitted on the same season's scores and are therefore in-sample descriptions, not held-out forecasts.

Inter vs Milan · Serie A 24/25	Inter	Milan
Season GF / GA / GD	79 / 35 / +44	61 / 43 / +18
Attack / defense rank	1 / 2	3 / 7
Direct meetings	Two recorded fixtures: 1–2 and 1–1; aggregate goals 2–3	

Transfermarkt goalkeeper conceded totals agree with independent scoreline totals in 366 of 388 covered club-seasons; clean sheets agree in 269 of 388. The latter coverage is insufficient for a league rank, so actual match results remain authoritative for comparison.

Application and interface

The primary interface is a static HTML/CSS/JavaScript editorial journal served by FastAPI. It is designed for browsing a dataset rather than presenting a single score. League and season filters are reflected in browser history and deep links. The app includes an overview, club directory/profile, two-club comparison, scenario Lab, source/methods information and a project evidence page.

Comparison behavior

Season comparison combines player-source attacking output with completed-match GF/GA/GD, points, league ranks and covered shot/SOT/corner totals. Head-to-head uses only meetings in the selected competition and season. Missing source coverage is a distinct state, not an empty 0–0 result. Match-level assists are not inferred from seasonal player totals.

Scenario Lab and API

The Lab prefills five real features for a selected club-season-competition, shows missing inputs, constrains attack spend to total spend, validates finite bounded inputs and compares baseline with an edited scenario. Browser requests are debounced; stale responses are discarded. No uncertainty interval is fabricated, and inputs outside observed ranges are identified.

Endpoint family	Role
`/api/health`, `/api/meta`	Readiness, available competitions/seasons and model summary
`/api/season`, `/api/analysis`	Bounded season rows and scoped analysis
`/api/clubs/*`, `/api/head-to-head`	Club history, defensive profile and direct meetings
`/api/lab/*`	Exact input configuration and validated inference
`/api/export`, `/api/project`	CSV export and model/source/system evidence

The interface has one visual theme. Club-style labels from optional K-Means are a separate exploratory analysis; they are neither a second UI style nor validated tactical categories.

Verification strategy and results

Validation covers the system at multiple layers. Unit and regression tests check source acquisition, match parsing, date and score validation, duplicate safety, target construction, feature lags, temporal split counts, missing values, output contracts and head-to-head aggregation. Fake HTTP responses exercise interrupted downloads and invalid hashes without relying on a live provider.

Live checks query the running API/database and compare output to source-dependent expectations. Browser checks use real Chrome in an isolated temporary profile and cover navigation, league filters, club comparison, Lab editing/reset, methods dialog, project evidence and narrow mobile widths. A separate defense-site browser check validates offline pages, persistent study checks, question search, timed rehearsal, keyboard-operated presentation, downloadable files and 390 px overflow.

Check group	Latest recorded result	What it establishes
Unit/regression	43 passed	Specified edge cases and contracts in the test suite
Live API/data	16 passed	Running prepared-data outputs and endpoint behavior
Application browser	15 passed	Core workflows and desktop/mobile interaction
Defense academy	9 passed	Offline materials, presentation and download integrity

These checks establish only the tested paths. They do not demonstrate bug-free software, independent contribution by multiple people, a second-laptop setup or external source permission. The current recorded test count is a dated project snapshot and should be refreshed after material code changes.

Health checks distinguish required outputs from saved-model presence. The app returns a not-ready response when required output tables are missing or empty; a missing optional match dataset is represented as unavailable coverage rather than falsified statistics.

Infrastructure measurement protocol

Performance evidence was collected on an Apple M3 Pro laptop with 18 GiB RAM and 12 logical CPUs. The Docker VM was configured for 12 CPUs and 7.70 GiB. Timings are arithmetic means of three repetitions, with individual runs retained. Query/index and full-system protocols were performed separately.

Measurement	Mean / observed value	Conditions
Cached source startup to API ready	31.10 s	Sources and Docker images already present
Loader stage	12.53 s	Verification, COPY, indices, optional enrichment
Pipeline stage	9.70 s	Aggregation, fitting and saved outputs
300 club lookups, no index	10.872 s	Deterministic sample and same query
300 lookups, indexed	0.075 s	145.4× faster for this query only
Database / local CSV footprint	695.5 / 356.6 MiB	Measured local snapshot

The lookup benchmark changes one factor: the selective index is absent or present. The result is specific to this query and sample; it does not mean every endpoint is 145 times faster. The frontend reads precomputed outputs, so this raw-table diagnostic is not the production response-time measurement.

Full-system runs preserve the database volume and source cache but reimport source rows and retrain the pipeline. OS/database caches are not flushed and concurrent host workloads are uncontrolled. The result is a local repeatable observation rather than a universal service-level guarantee.

Growth, memory and storage results

To observe scaling direction, the largest source table was loaded at 25%, 50% and 100% of its rows. Each level was repeated three times. The experiment changes data volume while retaining the same query and software conditions.

Table share	Rows	Load mean	Query mean
25%	469,679	1.40 s	0.297 s
50%	939,359	2.94 s	0.552 s
100%	1,878,719	6.18 s	0.998 s

Four times the row count corresponds to 4.40 times the load mean and 3.35 times the aggregation-query mean in this experiment. These are observed points, not a fitted scaling law. The query includes local preparation and aggregation work and should not be generalized to unrelated SQL.

Highest sampled container working set across three starts

Container	Observed maximum	Per-run sample counts
PostgreSQL	250.6 MiB	17 / 18 / 17
Loader	17.9 MiB	6 / 6 / 6
Pipeline	500.9 MiB	6 / 6 / 5
Application	107.3 MiB	2 / 2 / 2

Memory was sampled by `docker stats` at approximately two-second intervals. Short peaks between samples may be higher; values are not guaranteed instantaneous maxima. CSV inputs occupy 356.6 MiB and the resulting database including indices occupies 695.5 MiB in this snapshot.

Results and interpretation

The primary analysis does not support the simple claim that a larger incoming transfer budget generally improves next-season attacking output. The sample correlation is near zero. This finding does not imply transfers are irrelevant: fee reporting is incomplete, the outcome is narrow, and club strategies differ. It says only that the measured aggregate fee does not linearly track this target in the eligible sample.

Champagne Index is associated with completed same-season table position in the adequately covered sample. That is plausible as a retrospective descriptive result because goals and assists are related to scoring; it should not be presented as a surprise forecast or as evidence that the index causes a better finish. The index omits defensive performance, opponent quality and competition-specific points rules.

The temporal RandomForest beats zero change by about five percent on MAE while explaining a modest fraction of held-out variance. The augmented feature set improves both reported metrics slightly on the same holdout. This is enough to justify an interactive educational scenario, but not enough to claim reliable club decision support. A larger effect, external season holdout or independently reproduced sample would change that assessment.

Defensive statistics make the product comparison more complete: actual GA and GD, goals-per-match defensive ranks, observed shots against, and cross-checks against goalkeeper source totals. Their disagreement, especially for clean sheets, is a data-quality result rather than a reason to quietly choose the more flattering source.

> **Practical decision.** Present the tool as a data engineering and evaluation project that demonstrates careful source handling and a modest association/prediction exercise. Avoid claims about causal transfer value, measured xG, tactical superiority or production readiness.

Risks, limitations and responsible claims

Risk / limitation	Consequence	Current handling	Open action
Unconfirmed primary-source licence	Course's clear-licence condition not established	Source pinned; no raw-data redistribution; README caveat	Obtain permission or replace source
Undisclosed transfer fees	Recorded zeros are not reliable observed prices	State missingness and avoid literal spending conclusions	Seek fee coverage or narrow question
Champagne Index uses match proxy	Denominator may not equal verified fixtures	Define as source-match rate; distinguish scoreline data	Rebuild with fully matched fixture/player data
One temporal split	Score may depend on cutoff	Use season-grouped chronological holdout and baseline	Run rolling-origin/external evaluation
Outcome confounding	Associations do not identify interventions	Use descriptive language; expose denominators	Design a causal study only with stronger data and assumptions
No external deployment test	Another machine may expose setup gaps	Detailed README and checks	Independent fresh checkout on another laptop

The red-card analysis finds +0.341 conceded goals per team-match in observations with a sending-off (1,619 observations versus 16,195 without one). It is a same-match comparison; opponent strength, match state and red-card timing are confounders. The result is not a causal "red cards cost 0.341 goals" estimate.

No provider xG is present. Poisson ratings use the same season scores that they summarize. K-Means group names are centroid-derived exploratory labels. These three outputs are explicitly marked as proxies or in-sample descriptions to prevent a visual ranking from implying stronger evidence than the method supports.

Project evolution and contribution evidence

The repository history contains the initial schema, loader and pipeline work, followed by iterative corrections to source acquisition, target construction, evaluation, API behavior, comparisons and defense materials. The audit log records material defects and how they were corrected. Small work packages have acceptance conditions and corresponding test/check paths.

This repository is documented as a solo portfolio exercise. Git identities and role cards are not proof that multiple people independently implemented work. Existing branch history is preserved, and README reports author attribution as it appears in Git. New changes must be committed by the actual author using their authenticated account; changing a configured name or email is not evidence of contribution.

Work-package structure

Area	Responsibilities in the project	Review evidence
Data and SQL	Pinned sources, validation, schema, canonical aggregation	Source manifest, loader tests, SQL outputs
Analysis and evaluation	Targets, lags, temporal model, diagnostics	Protocol, baseline comparison, model output
Application and integration	API, frontend, comparison and run path	Live and browser checks, API docs
Infrastructure evidence	Repeatable startup, query and growth measurement	Three-run data and interpretation

The technical checklist is substantially implemented and measured. Human submission work remains: independent machine validation, source-owner permission/licence decision, actual presentation rehearsal and backup recording. A real group submission also requires the course’s actual membership and contributions; account-count balancing cannot satisfy that requirement.

For a group defense, participants should each make a substantive change or experiment, review another person’s work, and be able to explain the entire data flow. This report does not claim those actions occurred.

Conclusion

Calcio Champagne delivers an end-to-end local system for acquiring football data, validating and storing it, constructing club-season measures, evaluating a chronological prediction model and presenting results through an API and browser interface. The architecture stays inspectable: Compose defines service order, PostgreSQL stores source and derived tables, Python performs processing, and the frontend reads bounded outputs.

The most defensible empirical findings are narrow. Incoming recorded transfer spending has almost no linear association with the next consecutive season's change in this index in the eligible sample. A stronger same-season index/league-position relationship appears in adequately covered top-five club-seasons, but it is descriptive. The baseline temporal model improves modestly on predicting zero change, and adding defense/discipline/net-spend features together gives a further small improvement on the same held-out seasons.

Added scoreline and shot statistics improve comparison context while making source boundaries more visible. The project reports actual goals for and against and per-match ranks; it does not invent xG. SOT conversion and Poisson ratings are labeled proxies or in-sample descriptions. Infrastructure timings include repeated runs, hardware, sample sizes and measurement caveats rather than a single headline number.

The project is technically reproducible for its prepared local snapshot, subject to the documented legacy metadata difference and optional manual match files. It is not yet fully compliant with the course's open-source licensing requirement because the primary source licence remains unresolved. It also lacks independent second-machine validation and does not represent a real four-person contribution history.

> **Final assessment.** The strongest contribution is the complete, auditable data workflow and disciplined separation of measured results from descriptive proxies. The next meaningful step is not a stronger claim; it is resolving data rights and independently reproducing the run.

References

1. **Calcio Champagne repository.** Source code, tests, pipeline, SQL and report evidence. Branch snapshot: `codex/calcio-editorial-audit`. Key internal records: `README.md`, `docs/AUDIT.md`, `docs/ARCHITECTURE.md`, `docs/LAB.md`, `docs/MEASUREMENTS.md`, `docs/SOURCES.md`, `docs/COURSE_REQUIREMENTS.md`, `docs/WORK_PACKAGES.md`, `docs/findings.md`.
2. **Primary data snapshot:** salimt/football-datasets, pinned revision da9227e1f42baf74561324290dbf1ad2e3c00edd. Accessed and hash-checked 4 November 2026. The pinned revision has no confirmed explicit reuse licence.
3. **Match data:** football-data.co.uk, download page and fair-use policy. Local academic snapshot manually downloaded; match files are not redistributed.
4. **Course brief:** Vincenzo Lomonaco, *CIDS Lecture 15 — Project launch & architecture design*, 29 October 2026, 43-page PDF. Requirements are mapped by page in `docs/COURSE_REQUIREMENTS.md`; the source slides are not redistributed.
5. **Scikit-learn.** Time-series cross-validation guidance, used as reference for respecting temporal order: cross-validation and time-series split. This project uses a fixed chronological holdout, not a full rolling-origin evaluation.
6. **Poisson model implementation note.** The project fits a basic two-parameter attack/defense score model with shared home advantage. It omits the low-score correction of Dixon–Coles and is described as an in-sample rating, not as a reproduction of a commercial rating system.

All project measurements and local coverage figures in this report refer to the dated snapshot and execution conditions stated in the relevant section. Links were recorded for source identification; licensing remains the source owner’s responsibility.

Reproduction and appendix: command route

Fresh local run

```
git clone https://github.com/Ataika/Calcio-Champagne-prediction-automated-system
cd Calcio-Champagne-prediction-automated-system
cp .env.example .env
docker compose up --build
```

After startup, open `http://localhost:8502`. API documentation is at `/api/docs`. Docker Desktop and network access are needed for the initial image/source acquisition; already verified CSVs can be reused offline. The 31.10 s result applies only to cached-source and cached-image measurements, not a fresh download/build.

Verification commands

```
docker compose run --rm --no-deps -v "$PWD:/source:ro" -w /source app   python -m unittest d
python3 scripts/check_live.py
node scripts/browser_check.mjs
```

Browser checks need local Chrome and Node 22+. Run database-dependent checks only after the Compose stack is healthy. The measure tool is separate and on demand; it writes local results under `measure/results/`.

Repository map

Path	Purpose
`loader/`, `sql/`	Source acquisition, schema and canonical query logic
`pipeline/`, `models/`	Derived metrics, evaluation and local inference artifact
`app/`	FastAPI service and browser interface
`measure/`, `scripts/`	Repeatable infrastructure and application checks
`docs/`, `defense/`	Methods, provenance, audit and preparation materials

Data handling: do not place credentials in Git, do not redistribute the prepared match CSVs, and resolve the primary source licence before claiming open-data compliance. The model artifact is rebuilt by the pipeline and is not included in the project archive.

End of report · Calcio Champagne · Evidence snapshot 6 November 2026